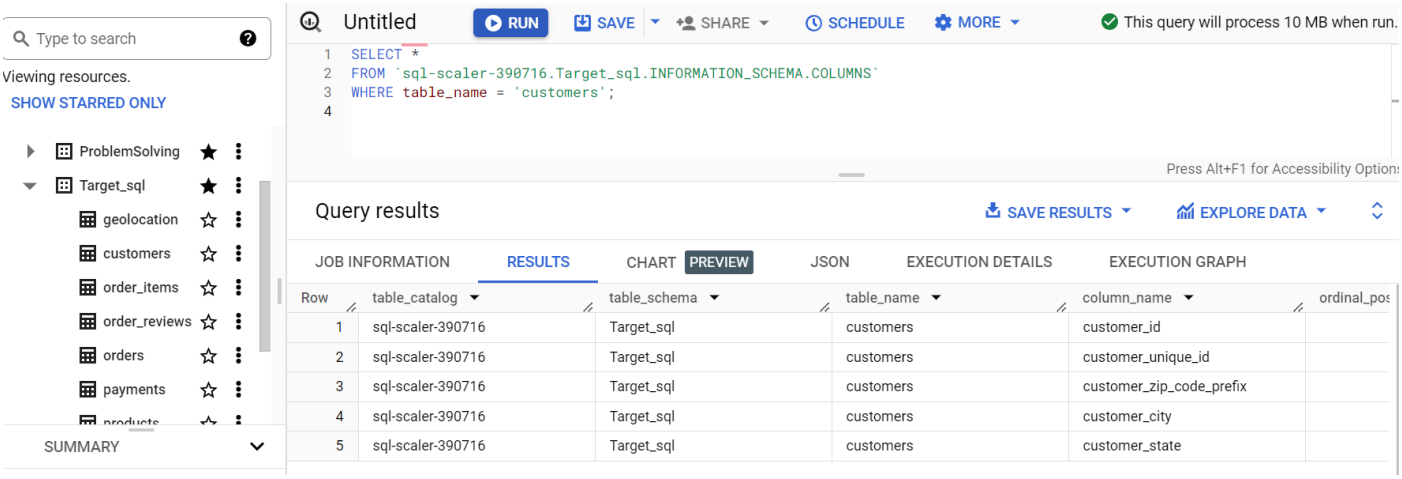
1. **Import the dataset and do the usual exploratory analysis steps like checking the structure and characteristics of the dataset:**
   a)Data type of all columns in the "customers" table.
   b)Get the time range between which the orders were placed.
   c)Count the Cities and states of customers who ordered during the given period.

### 1a) Query:

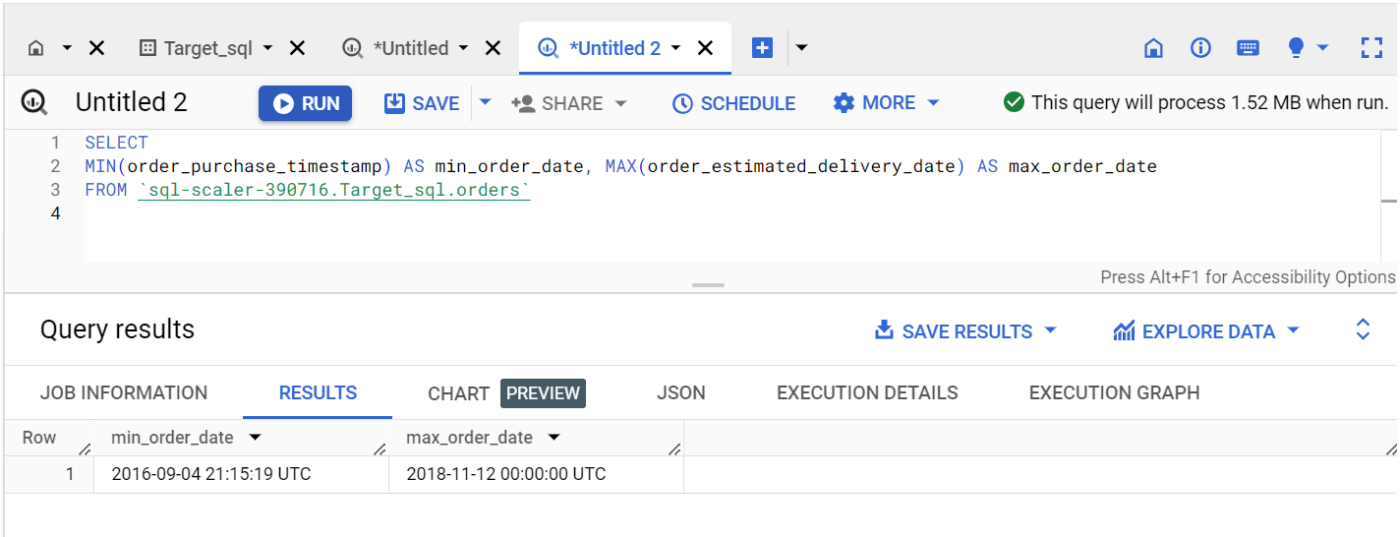
```
SELECT *
FROM `sql-scaler-390716.Target_sql.INFORMATION_SCHEMA.COLUMNS`
WHERE table_name = 'customers';
```




| Row | table_catalog | table_schema | table_name | column_name | ordinal_pos |
|---|---|---|---|---|---|
| 1 | sql-scaler-390716 | Target_sql | customers | customer_id | |
| 2 | sql-scaler-390716 | Target_sql | customers | customer_unique_id | |
| 3 | sql-scaler-390716 | Target_sql | customers | customer_zip_code_prefix | |
| 4 | sql-scaler-390716 | Target_sql | customers | customer_city | |
| 5 | sql-scaler-390716 | Target_sql | customers | customer_state | |

### 1b) Query:

```
SELECT
MIN(order_purchase_timestamp) AS min_order_date, MAX(order_estimated_delivery_date)
AS max_order_date
FROM `sql-scaler-390716.Target_sql.orders`
```

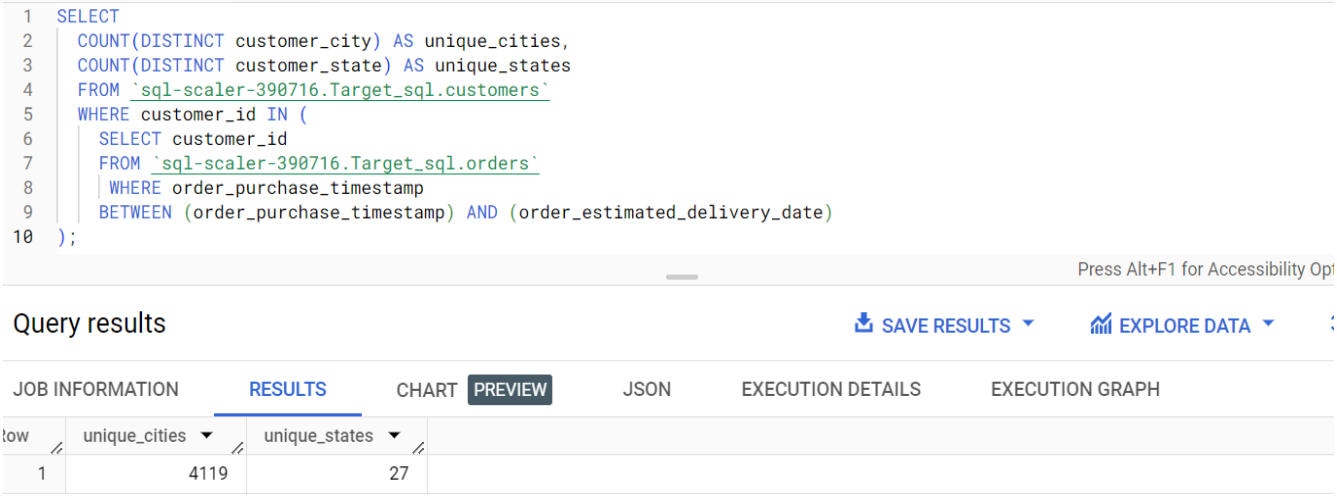



| Row | min_order_date | max_order_date |
|---|---|---|
| 1 | 2016-09-04 21:15:19 UTC | 2018-11-12 00:00:00 UTC |

**1c) Query:**

```
SELECT
  COUNT(DISTINCT customer_city) AS unique_cities,
  COUNT(DISTINCT customer_state) AS unique_states
  FROM `sql-scaler-390716.Target_sql.customers`
  WHERE customer_id IN (
    SELECT customer_id
    FROM `sql-scaler-390716.Target_sql.orders`
     WHERE order_purchase_timestamp
    BETWEEN (order_purchase_timestamp) AND (order_estimated_delivery_date)
);
```

IN Between we need to mention start and end date if given for a  period of time

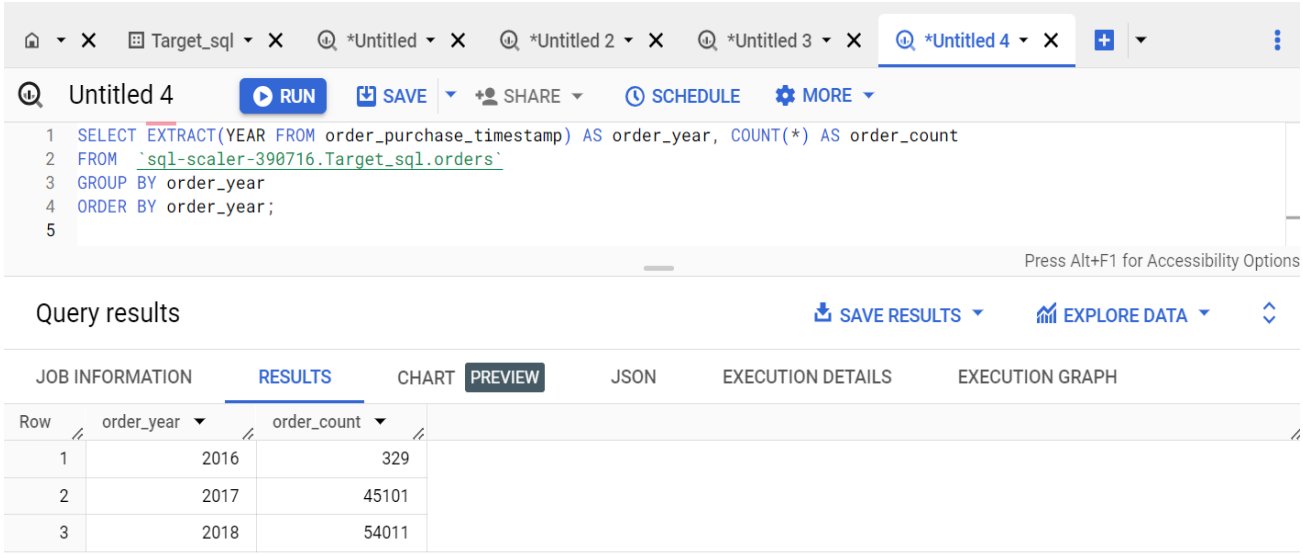



## 2. In-depth Exploration:

1. Is there a growing trend in the no. of orders placed over the past years?
2. Can we see some kind of monthly seasonality in terms of the no. of orders being placed?
3. During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)
    - 0-6 hrs: Dawn
    - 7-12 hrs: Mornings
    - 13-18 hrs: Afternoon
    - 19-23 hrs: Night

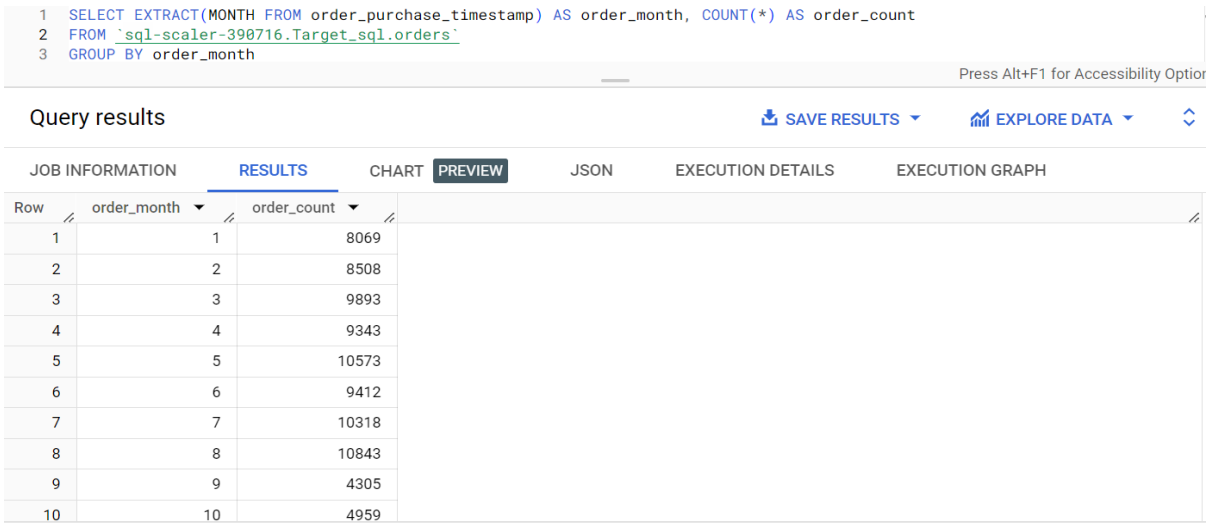
**2a) Query:**

```
SELECT EXTRACT(YEAR FROM order_purchase_timestamp) AS order_year,
COUNT(*) AS order_count
FROM  `sql-scaler-390716.Target_sql.orders`
GROUP BY order_year
ORDER BY order_year;
```




**2b)Query:**

```
SELECT EXTRACT(MONTH FROM order_purchase_timestamp) AS order_month, COUNT(*) AS order_count
FROM `sql-scaler-390716.Target_sql.orders`
GROUP BY order_month
ORDER BY order_month;
```




**2c) Query:**

```
SELECT
    CASE
        WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 0 AND 6 THEN
'Dawn'
        WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 7 AND 12 THEN
'Morning'
        WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 13 AND 18 THEN
'Afternoon'
        ELSE 'Night'
    END AS order_time_of_day,
    COUNT(*) AS order_count
FROM
    `sql-scaler-390716.Target_sql.orders`

WHERE customer_id IN (
    SELECT customer_id FROM `sql-scaler-390716.Target_sql.customers`
```

```
        WHERE customer_state='Brazil'
    )

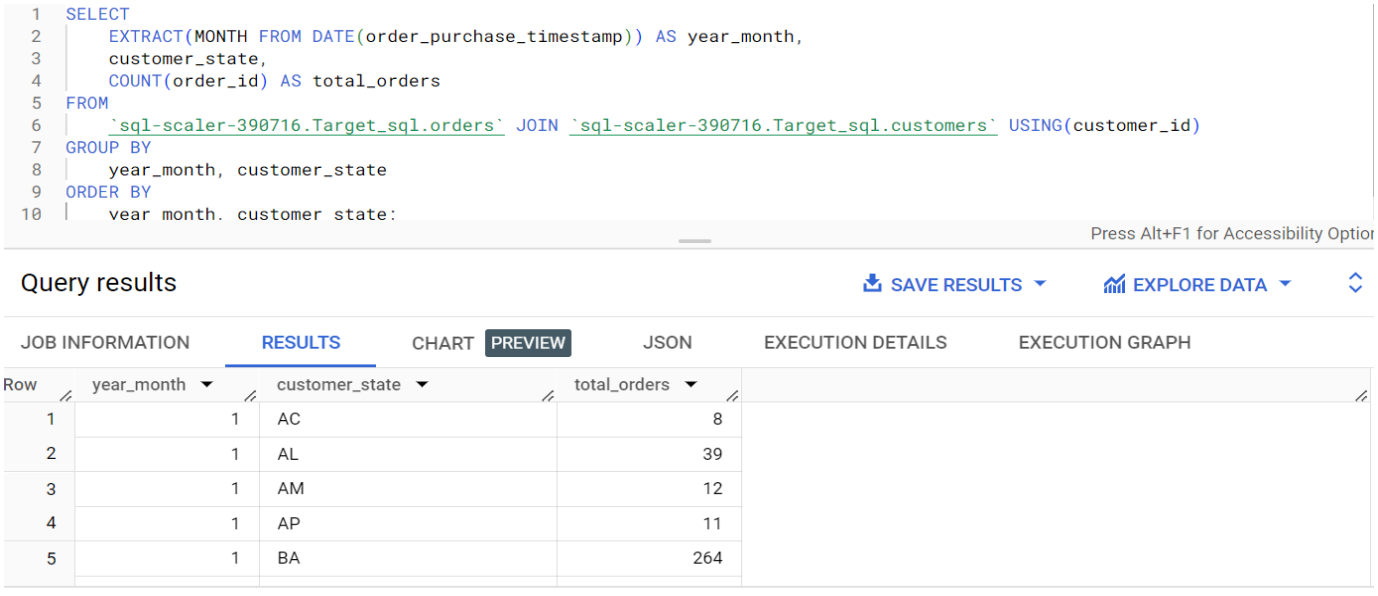
    GROUP BY
        order_time_of_day
    ORDER BY
        order_time_of_day;
```

NOTE: I couldn't find the Brazil code in customer_state in the provided Dataset

3. **Evolution of E-commerce orders in the Brazil region:**
   a) Get the month-on-month no. of orders placed in each state.
   b)How are the customers distributed across all the states?

**3a) Query:**

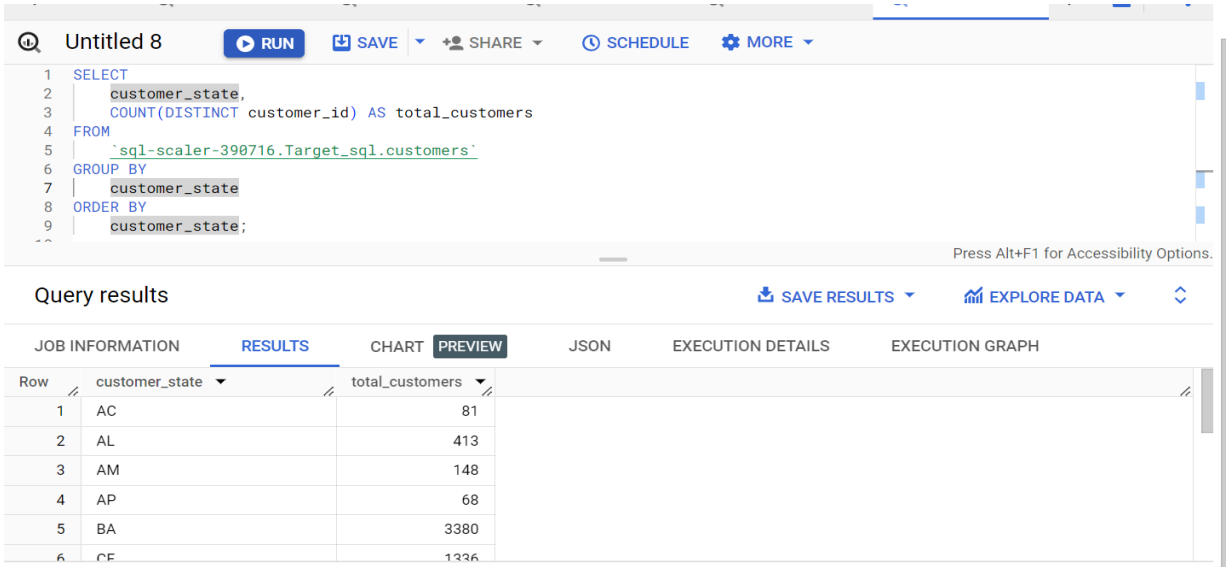
```
SELECT
    EXTRACT(MONTH FROM DATE(order_purchase_timestamp)) AS year_month,
    customer_state,
    COUNT(order_id) AS total_orders
FROM
    `sql-scaler-390716.Target_sql.orders` JOIN `sql-scaler-390716.Target_sql.customers`
USING(customer_id)
GROUP BY
    year_month, customer_state
ORDER BY
    year_month, customer_state;
```




**3b) Query:**

```
    SELECT
        customer_state,
        COUNT(DISTINCT customer_id) AS total_customers
    FROM
        `sql-scaler-390716.Target_sql.customers`
    GROUP BY
        customer_state
```

```
ORDER BY
    customer_state;
```




4 . **Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.**

1. Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).
   You can use the "payment_value" column in the payments table to get the cost of orders.
2. Calculate the Total & Average value of order price for each state.
3. Calculate the Total & Average value of order freight for each state.

**4a) Query :**

**4b) Query:**

```
SELECT
    customer_state,
    SUM(payment_value) AS total_order_price,
    AVG(payment_value) AS average_order_price
FROM
    `sql-scaler-390716.Target_sql.payments` p
JOIN
`sql-scaler-390716.Target_sql.orders` o
ON
    p.order_id = o.order_id
    JOIN
    `sql-scaler-390716.Target_sql.customers` c

ON
    o.customer_id = c.customer_id

GROUP BY
    customer_state;
```

```
SELECT
    customer_state,
    SUM(payment_value) AS total_order_price,
    AVG(payment_value) AS average_order_price
FROM
    `sql-scaler-390716.Target_sql.payments` p
JOIN
`sql-scaler-390716.Target_sql.orders` o
```

Press Alt+F1 for Accessibility Optic

Query results — SAVE RESULTS — EXPLORE DATA

JOB INFORMATION | RESULTS | CHART PREVIEW | JSON | EXECUTION DETAILS | EXECUTION GRAPH

| Row | customer_state | total_order_price | average_order_price |
|---|---|---|---|
| 1 | RJ | 2144379.689999... | 158.5258882235... |
| 2 | RS | 890898.5399999... | 157.1804057868... |
| 3 | SP | 5998226.959999... | 137.5046297739... |
| 4 | DF | 355141.0800000... | 161.1347912885... |
| 5 | PR | 811156.3799999... | 154.1536259977... |
| 6 | MT | 187029.2900000... | 195.2289039665... |
| 7 | MA | 152523.0200000... | 198.8566101694... |

4c) Query:

```
SELECT customer_state,
       SUM(freight_value) AS total_freight,
       AVG(freight_value) AS average_freight
FROM `sql-scaler-390716.Target_sql.order_items` oi
JOIN `sql-scaler-390716.Target_sql.orders` o ON oi.order_id = o.order_id
JOIN `sql-scaler-390716.Target_sql.customers` c ON c.customer_id= o.customer_id
GROUP BY customer_state;
```

Untitled 11 — RUN — SAVE — SHARE — SCHEDULE — MORE

```
SELECT customer_state,
       SUM(freight_value) AS total_freight,
       AVG(freight_value) AS average_freight
FROM `sql-scaler-390716.Target_sql.order_items` oi
JOIN `sql-scaler-390716.Target_sql.orders` o ON oi.order_id = o.order_id
JOIN `sql-scaler-390716.Target_sql.customers` c ON c.customer_id= o.customer_id
GROUP BY customer_state;
```

Press Alt+F1 for Accessibility Opt

Query results — SAVE RESULTS — EXPLORE DATA

JOB INFORMATION | RESULTS | CHART PREVIEW | JSON | EXECUTION DETAILS | EXECUTION GRAPH

| Row | customer_state | total_freight | average_freight |
|---|---|---|---|
| 1 | MT | 29715.43000000... | 28.16628436018... |
| 2 | MA | 31523.77000000... | 38.25700242718... |
| 3 | AL | 15914.58999999... | 35.84367117117... |
| 4 | SP | 718723.0699999... | 15.14727539041... |
| 5 | MG | 270853.4600000... | 20.63016680630... |
| 6 | PE | 59449.65999999... | 32.91786267995... |
| 7 | RJ | 305589.3100000... | 20.96092393168... |

## 5 Analysis based on sales, freight, and delivery time.

1.)Find the no. of days taken to deliver each order from the order's purchase date as delivery time.

Also, calculate the difference (in days) between the estimated & actual delivery date of an order.

Do this in a single query.

You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:

**time_to_deliver** = order_delivered_customer_date - order_purchase_timestamp

**diff_estimated_delivery** = order_estimated_delivery_date - order_delivered_customer_date

2.Find out the top 5 states with the highest & lowest average freight value.

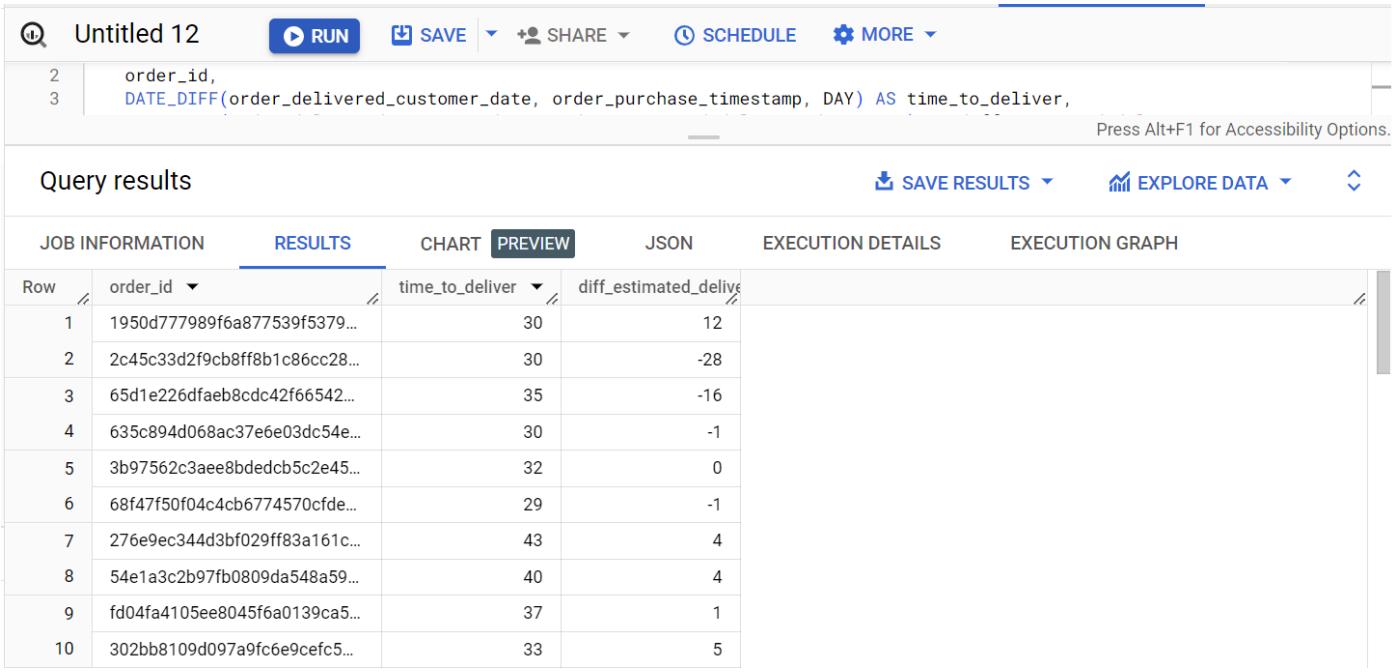
3.Find out the top 5 states with the highest & lowest average delivery time.

4.Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

**5a) Query:**

```
SELECT
    order_id,
    DATE_DIFF(order_delivered_customer_date, order_purchase_timestamp, DAY) AS
time_to_deliver,
    DATE_DIFF(order_delivered_customer_date, order_estimated_delivery_date, DAY) AS
diff_estimated_delivery
FROM
    `sql-scaler-390716.Target_sql.orders`
WHERE
    order_delivered_customer_date IS NOT NULL
    AND order_estimated_delivery_date IS NOT NULL;
```




| Row | order_id | time_to_deliver | diff_estimated_delive |
|---|---|---|---|
| 1 | 1950d777989f6a877539f5379... | 30 | 12 |
| 2 | 2c45c33d2f9cb8ff8b1c86cc28... | 30 | -28 |
| 3 | 65d1e226dfaeb8cdc42f66542... | 35 | -16 |
| 4 | 635c894d068ac37e6e03dc54e... | 30 | -1 |
| 5 | 3b97562c3aee8bdedcb5c2e45... | 32 | 0 |
| 6 | 68f47f50f04c4cb6774570cfde... | 29 | -1 |
| 7 | 276e9ec344d3bf029ff83a161c... | 43 | 4 |
| 8 | 54e1a3c2b97fb0809da548a59... | 40 | 4 |
| 9 | fd04fa4105ee8045f6a0139ca5... | 37 | 1 |
| 10 | 302bb8109d097a9fc6e9cefc5... | 33 | 5 |

**5b) Query:**

```
WITH cte as
(
SELECT customer_state,
       AVG(freight_value) AS average_freight
FROM `sql-scaler-390716.Target_sql.order_items` oi
JOIN `sql-scaler-390716.Target_sql.orders` o ON oi.order_id = o.order_id
JOIN `sql-scaler-390716.Target_sql.customers` c ON c.customer_id= o.customer_id
```

```
        GROUP BY customer_state
        )
        SELECT
        cte.customer_state,
        cte.average_freight
        FROM cte
        ORDER BY cte.average_freight DESC
        LIMIT 5

        UNION ALL

        SELECT
        cte.customer_state,
        cte.average_freight
        FROM cte
        ORDER BY cte.average_freight ASC
        LIMIT 5;
```

## 5c) Query:

```
            WITH CTE AS (

    SELECT
        customer_state,
        AVG(DATE_DIFF(order_delivered_customer_date, order_purchase_timestamp, DAY)) AS
avg_delivery_time
    FROM
        `sql-scaler-390716.Target_sql.orders` o
        JOIN `sql-scaler-390716.Target_sql.customers` c
        ON o.customer_id = c.customer_id

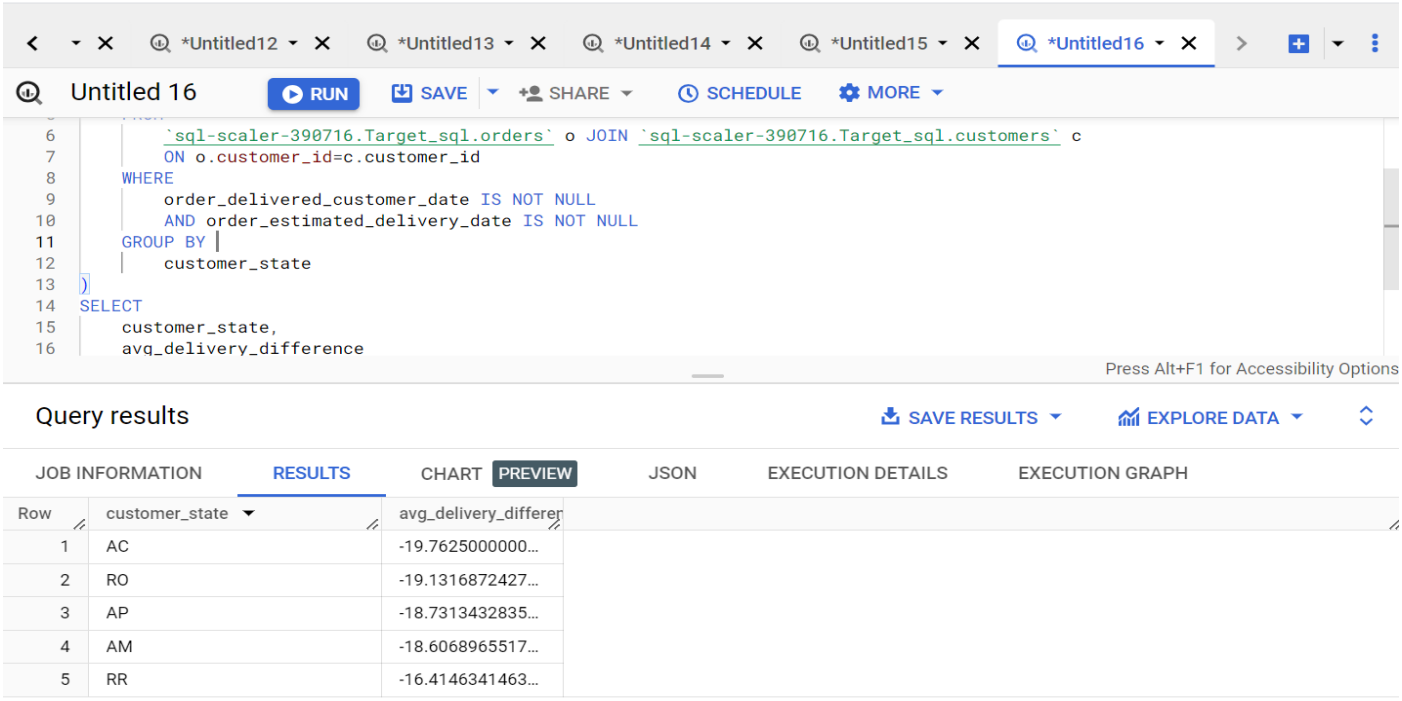
    WHERE
        order_delivered_customer_date IS NOT NULL
    GROUP BY
        customer_state
)
SELECT
    CTE.customer_state,
    CTE.avg_delivery_time
FROM
    CTE
ORDER BY
    avg_delivery_time DESC
LIMIT 5

UNION ALL

SELECT
    CTE.customer_state,
    CTE.avg_delivery_time
FROM
    CTE
ORDER BY
    avg_delivery_time ASC
LIMIT 5;
```

**5d) Query:**

```
WITH CTE  AS (
    SELECT
        customer_state,
        AVG(DATE_DIFF(order_delivered_customer_date, order_estimated_delivery_date,
DAY)) AS avg_delivery_difference
    FROM
        `sql-scaler-390716.Target_sql.orders` o JOIN `sql-scaler-
390716.Target_sql.customers` c
        ON o.customer_id=c.customer_id
    WHERE
        order_delivered_customer_date IS NOT NULL
        AND order_estimated_delivery_date IS NOT NULL
    GROUP BY
        customer_state
)
SELECT
    customer_state,
    avg_delivery_difference
FROM
    CTE
ORDER BY
    avg_delivery_difference ASC
LIMIT 5;
```



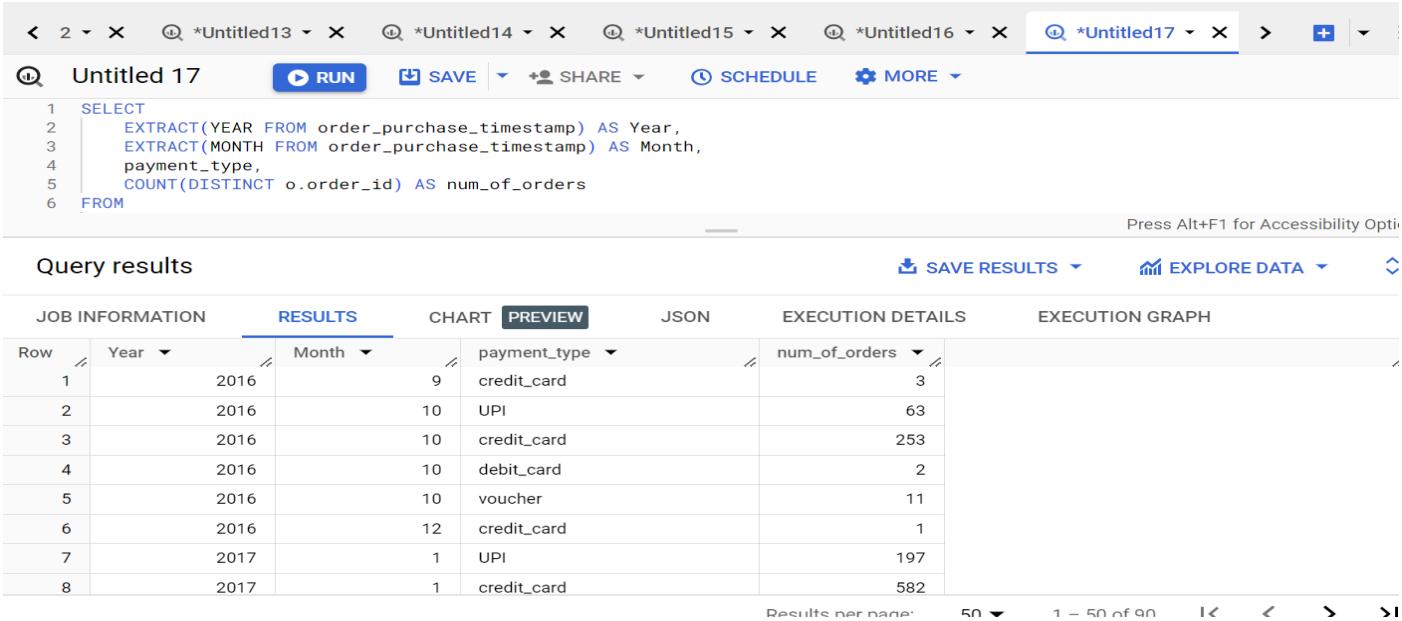

6. **Analysis based on the payments:**

a) Find the month-on-month no. of orders placed using different payment types.

b) Find the no. of orders placed on the basis of the payment installments that have been paid.

## 6a) Query:

```sql
SELECT
    EXTRACT(YEAR FROM order_purchase_timestamp) AS Year,
    EXTRACT(MONTH FROM order_purchase_timestamp) AS Month,
    payment_type,
    COUNT(DISTINCT o.order_id) AS num_of_orders
FROM
    `sql-scaler-390716.Target_sql.orders` o JOIN `sql-scaler-390716.Target_sql.payments` p
    ON o.order_id = p.order_id
GROUP BY
    Year, Month, payment_type
ORDER BY
    Year, Month, payment_type;
```




## 6b)Query:

```sql
SELECT
    payment_installments,
    COUNT(DISTINCT o.order_id) AS num_of_orders
FROM
   `sql-scaler-390716.Target_sql.orders` o JOIN `sql-scaler-390716.Target_sql.payments` p
   ON o.order_id=p.order_id
GROUP BY
    payment_installments
ORDER BY
    payment_installments;
```

Untitled 18 RUN SAVE SHARE SCHEDULE MORE

```
6   ON o.order_id=p.order_id
7 GROUP BY
8   payment_installments
```

Press Alt+F1 for Accessibility

## Query results

SAVE RESULTS EXPLORE DATA

JOB INFORMATION | RESULTS | CHART PREVIEW | JSON | EXECUTION DETAILS | EXECUTION GRAPH

| Row | payment_installment | num_of_orders |
|---|---|---|
| 1 | 0 | 2 |
| 2 | 1 | 49060 |
| 3 | 2 | 12389 |
| 4 | 3 | 10443 |
| 5 | 4 | 7088 |
| 6 | 5 | 5234 |
| 7 | 6 | 3916 |
| 8 | 7 | 1623 |
| 9 | 8 | 4253 |
| 10 | 9 | 644 |